Handout

PV

UML / Klassendiagramm

WBS TRAINING AG
Lorenzweg 5
D-12099 Berlin
Amtsgericht Berlin HRB 68531
Sitz der Gesellschaft: Berlin

Vorstand:
Heinrich Kronbichler,
Joachim Giese
Aufsichtsrat (Vorsitz): Dr. Daniel Stadler
USt-IdNr.: DE 209 768 248

GLS Gemeinschaftsbank eG
IBAN: DE18 4306 0967 1146 1814 00
BIC: GENODEM1GLS

DQS-zertifiziert nach
DIN EN ISO 9001:2015 Reg.-Nr. 015344 QM15
Zulassung nach AZAV Reg.-Nr. 015344 AZAV

WBS TRAINING

Inhalt

1 Klassendiagramm 2
1.1 Übersicht 2
1.1.1 Erklärung / Definition 2
1.2 Syntax 3
1.2.1 Die Klasse 3
1.2.2 Das Interface 5
1.2.3 Das Enum 5
1.3 Beziehungen 6
1.3.1 Assoziation 6
1.3.2 Aggregation 7
1.3.3 Komposition 7
1.3.4 Vererbung 8
1.3.5 Implementierung 8
1.3.6 Weitere 9
1.3.7 IHK-Prüfung 9
1.4 Syntax Zusammenfassung 10
1.5 Übungsaufgabe 11
1.5.1 Aufgabenstellung 11
1.5.2 Lösung 12

1 Klassendiagramm

1.1 Übersicht

1.1.1 Erklärung / Definition

Ein UML-Klassendiagramm ist ein visuelles Werkzeug, das in der Unified Modeling Language (UML) verwendet wird. Es stellt Klassen, Schnittstellen und ihre Beziehungen dar, um ein System zu beschreiben. Es visualisiert die verschiedenen Objekttypen und ihre statischen Beziehungen innerhalb eines Systems.

In einem Klassendiagramm werden Klassen durch Rechtecke dargestellt, die den Namen der Klasse, ihre Attribute und Operationen enthalten können. Diese drei Elemente sind durch horizontale Linien getrennt. Das Diagramm hilft dabei, die Struktur eines Systems zu analysieren und zu verstehen.

Kurz gesagt:

Das UML-Klassendiagramm wird verwendet, um die Klassen in einem Programm und deren Beziehungen zueinander zu dokumentieren und Entwicklern eine Übersicht zu geben.

Beispiel 1:

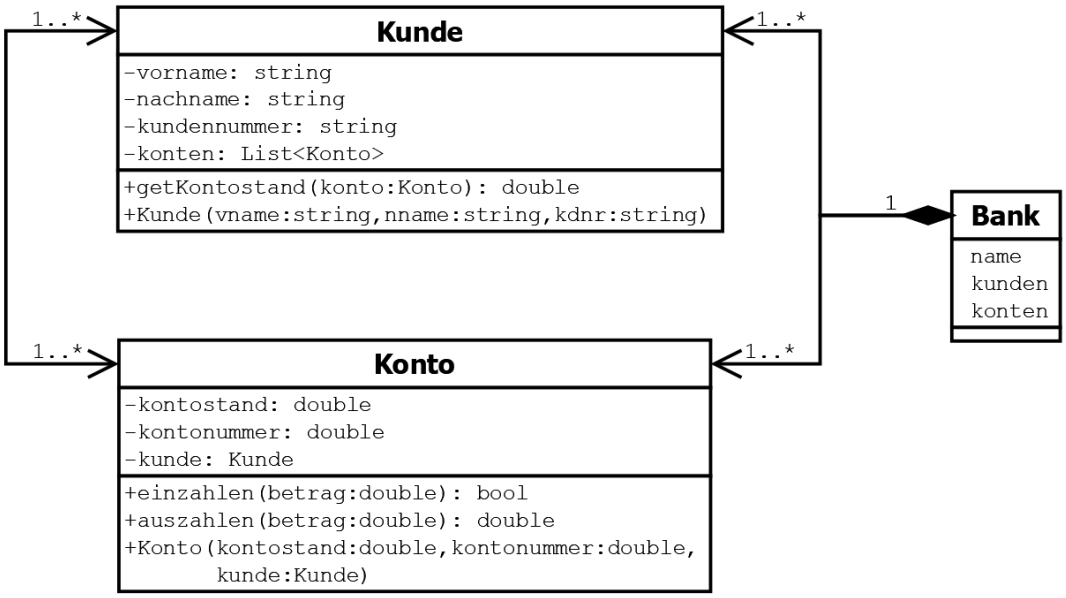


Abb. 1 Darstellung -Klassendiagramm (Bank-Kunde-Konto)

Die Darstellung zeigt ein UML-Klassendiagramm, das die Beziehungen zwischen einer Bank, ihren Kunden und deren Konten repräsentiert. Es besteht aus drei Klassen: “Kunde”, “Konto” und “Bank”.

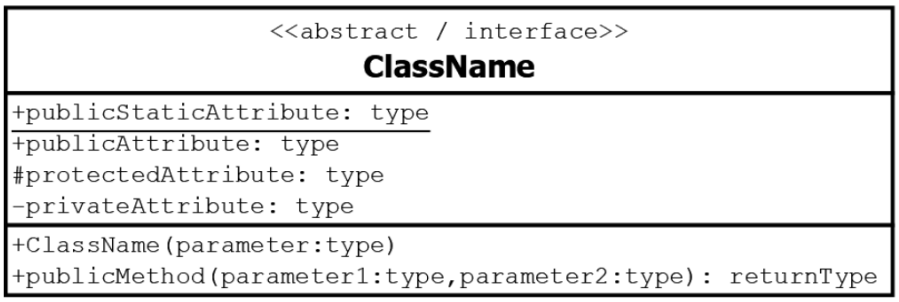
- Die Klasse *Kunde* hat Attribute wie Vorname, Nachname, Kundennummer und eine Liste von Konten. Sie hat auch Methoden, um den Kontostand abzurufen und einen neuen Kunden zu erstellen.
- Die Klasse *Konto* enthält Attribute wie Kontostand und Kontonummer. Sie hat Methoden zum Ein- und Auszahlen von Geld sowie einen Konstruktor zum Erstellen eines Kontos.
- Die Klasse *Bank* ist mit beiden, “Kunde” und “Konto”, verbunden, was darauf hinweist, dass sie Informationen über Kunden und deren Konten enthält.

Die Beziehungen sind wie folgt:

- Eine Bank kann mehrere Kunden haben (1…*).
- Ein Kunde kann mehrere Konten haben (1…*).
- Ein Konto ist mit einem Kunden verbunden, könnte aber potenziell über den Kunden mit mehreren Banken verknüpft sein.

1.2 Syntax

1.2.1 Die Klasse



Die Klasse ist in 3 Bereiche eingeteilt:

Klassenname: Dieser steht im oberen Bereich des Rechtecks und beginnt immer mit einem Großbuchstaben, optional dazu der Stereotyp

Attribute: Sie repräsentieren die Merkmale der Objekte und werden im mittleren Bereich des Rechtecks aufgelistet.

Operationen: Sie stellen das Verhalten der Klasse dar und listen die Methoden auf, die die Objekte ausführen können. Sie werden im unteren Bereich des Rechtecks aufgeführt.

Attribute und Methoden werden zusammen als die „Member der Klasse“ bezeichnet

Stereotyp:

- <<abstract>> bei abstrakten Klassen
- <<interface>> bei Interfaces
- <<struct>> bei Strukturen
- <<enum>> für Enumeratoren

Syntax für Attribute:

Sichtbarkeit Attributname : Typ {Eigenschaften}

Syntax für Methoden:

Sichtbarkeit Methodenname(Parameterliste) : Rückgabetyp {Eigenschaften}

Parameter:

Parametername : Typ

Sichtbarkeit:

+ public

protected

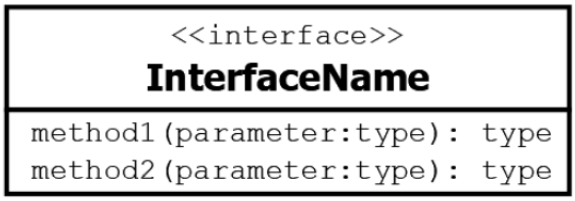
- private

~ package

Eigenschaften:

{static, final, abstract, …}

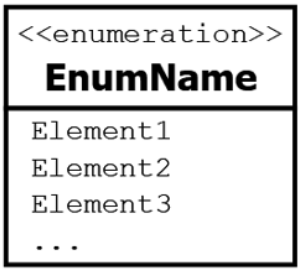
1.2.2 Das Interface



Im Unterschied zu Klassen, werden Interfaces[1] häufig nur mit zwei Bereichen dargestellt. Kopf und Methoden, drei Bereiche sind aber nicht unüblich (gerade bei IHK-Aufgaben kommt das vor)

Wie bereits erwähnt, werden Interfaces mit <<interface>> ausgezeichnet. Interface-Member haben keine Zugriffsmodifizierer/Sichtbarkeiten (manchmal sind sie aber auch einfach public)

1.2.3 Das Enum



In einem UML-Klassendiagramm wird ein Enum[2] (Enumeration) als Rechteck dargestellt, das seine möglichen Werte auflistet. Der Name des Enums steht oben, während die einzelnen Werte darunter aufgelistet sind. Um es von anderen Elementen zu unterscheiden, wird das Rechteck mit dem Stereotyp «enumeration» gekennzeichnet. Ein Enum kann auch mit einer

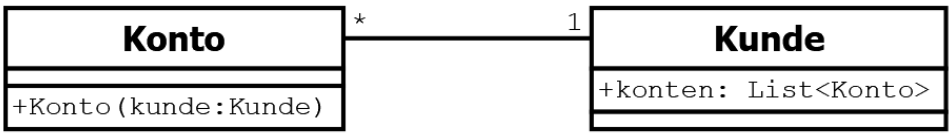
[1] In der Programmierung ist ein Interface (oder Schnittstelle) ein Konstrukt, das die Struktur des Codes definiert. Es legt die Methoden fest, die eine Klasse implementieren muss, und dient als Standard für die Kommunikation zwischen zwei Parteien. Wenn eine Klasse ein Interface implementiert, muss sie alle Methoden mit den entsprechenden Signaturen bereitstellen. Interfaces ermöglichen die Austauschbarkeit von Instanzen unabhängig von ihrem Typ und trennen den Kontrakt von der Implementierung. Es ist zu beachten, dass die Implementierung von Interfaces in verschiedenen Programmiersprachen variiert.

[2] Ein Enum (Enumeration) ein spezieller Datentyp, der eine Reihe von benannten Konstanten definiert. Enums verbessern die Lesbarkeit und Wartbarkeit des Codes, da jeder Konstante ein Name gegeben wird. Es ist zu beachten, dass die Implementierung von Enums in Java von anderen Programmiersprachen abweichen kann.

Klasse verbunden sein, um anzuzeigen, dass ein Attribut, eine Operation oder ein Parameter den Enum als Typ verwendet.

1.3 Beziehungen

1.3.1 Assoziation



Sobald eine Klasse Attribute vom Typ einer anderen Klasse (oder Interface, Struct, Enum, etc.) besitzt, oder diese andere Klasse in Methoden verwendet, liegt eine Assoziation vor. Die beiden Klassen stehen in einer Beziehung zueinander.

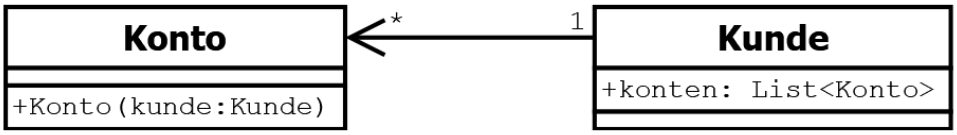
Ähnlich wie bei Datenbank-Diagrammen, werden hier Multiplizitäten (Bei DB-Kardinalität genannt) angegeben.

`*` = beliebig viele

`1..*` = mindestens 1

`2..5` = mindestens 2, höchstens 5

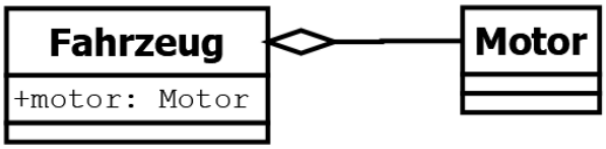
`1` = genau 1



Assoziationen können auch Pfeilspitzen haben. Hierbei handelt es sich dann um eine gerichtete Assoziation. Der Pfeil zeigt auf die Klasse, auf die navigiert werden kann. Habe ich ein Objekt von Kunde, kann ich über Kunde nun auf die Konto-Objekte navigieren und z.B. von Konto Methoden aufrufen.

Assoziationen können auch beschriftet werden und gelegentlich haben sie auch Pfeile, die die Leserichtung angeben.

1.3.2 Aggregation

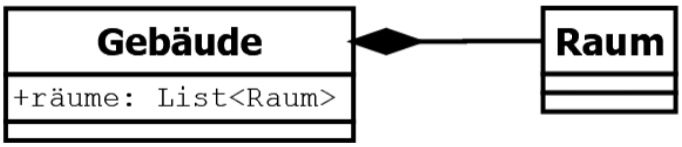


Ein Spezialfall von Assoziation ist die Aggregation.

Sobald eine Klasse aus Objekten einer anderen Klasse besteht, ist es eine Aggregation. Hier sprechen wir von „hat“, „ist Teil von“ oder „besteht aus“. Das Fahrzeug hat einen Motor. Damit liegt eine Aggregation vor.

Die Navigierbarkeit ist hierbei in der Regel vom Ganzen auf die Teile (also hier von Fahrzeug auf Motor).

1.3.3 Komposition

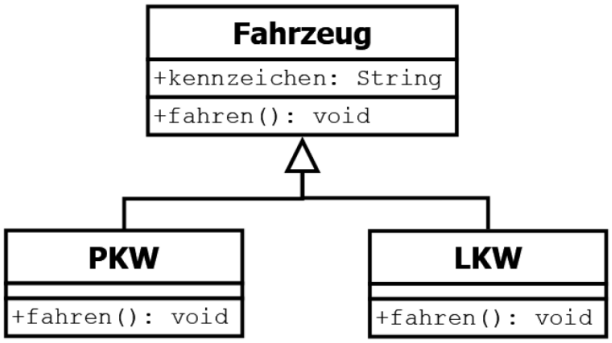


Ein weiterer Spezialfall ist die Komposition.

Es gelten dieselben Bedingungen wie bei einer Aggregation, der Unterschied liegt aber darin, dass bei einer Komposition eine existentielle Abhängigkeit vorliegt. Das eine kann nicht ohne das andere existieren.

Ein Raum kann nicht ohne ein passendes Gebäude existieren. Werden alle Teile gelöscht, wird auch das Ganze gelöscht. Wird das Ganze gelöscht, verschwinden auch alle Teile.

1.3.4 Vererbung

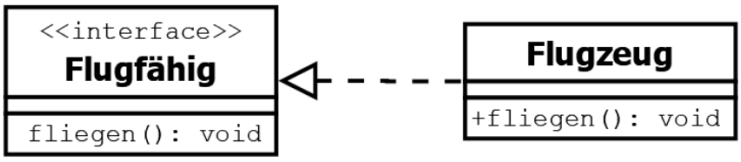


Klassen können voneinander erben. Es wird auch von Generalisierung/Spezialisierung gesprochen. Hier liegt eine „ist “-Beziehung vor. Ein PKW IST EIN Fahrzeug. Ein LKW IST EIN Fahrzeug.

Wichtig: Nicht jedes Fahrzeug ist auch automatisch ein PKW oder LKW!

Member, außer Konstruktoren, werden von der Eltern-Klasse (auch Basis-Klasse oder Super-Klasse genannt) an die Kind-Klassen (auch Sub-Klasse genannt) vererbt. Im Klassendiagramm werden geerbte Member nicht nochmal wiederholt, außer sie werden überschrieben, zum Beispiel wenn der Member der Basis-Klasse abstract ist oder man möchte ein neues Verhalten implementieren.

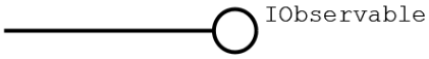
1.3.5 Implementierung



Interfaces werden nicht geerbt, sondern implementiert. Es wird gelegentlich auch von Realisierung gesprochen. Wie bei der Vererbung handelt es sich hier um eine „ist“-Beziehung. Das Flugzeug IST Flugfähig, da es das Interface Flugfähig implementiert.

Im Klassendiagramm müssen alle Nicht-Standard-Schnittstellenmember in der implementierenden Klasse wiederholt werden.

1.3.6 Weitere



Eine Klasse implementiert eine externe, nicht genauer spezifizierte Schnittstelle.

Abhängigkeit: Die Klasse wird nicht als Attribut (Assoziation) verwendet, sondern als Objekte zum Beispiel durch Methoden erzeugt.

1.3.7 IHK-Prüfung

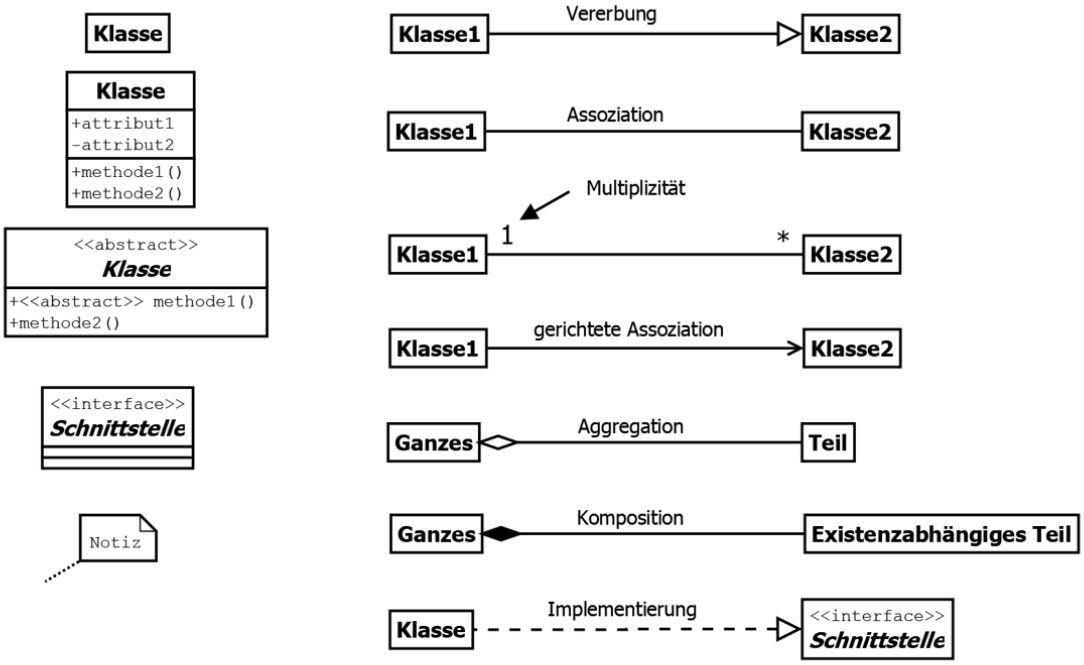
In der IHK-Prüfung werden Klassendiagramme gerne zusammen mit Entwurfsmustern abgefragt.

Entwurfsmuster sind beispielsweise:

- Strategy
- Observer
- Adapter
- Factory
- Decorator

Dabei geht es aber häufig nur darum, korrekt Methoden und Beziehungen einzutragen. Zum Üben von Klassendiagrammen (und auch den anderen UML Diagrammen) empfehle ich, alte IHK Prüfungen zu bearbeiten.

1.4 Syntax Zusammenfassung



Syntax für Attribute:

Sichtbarkeit Attributname : Typ {Eigenschaften}

Syntax für Methoden:

Sichtbarkeit Methodenname(parameter1 : Typ, ...) : Rückgabetyp {Eigenschaften}

Sichtbarkeit:

+ public

protected

- private

Eigenschaften:

{static, final, ...}

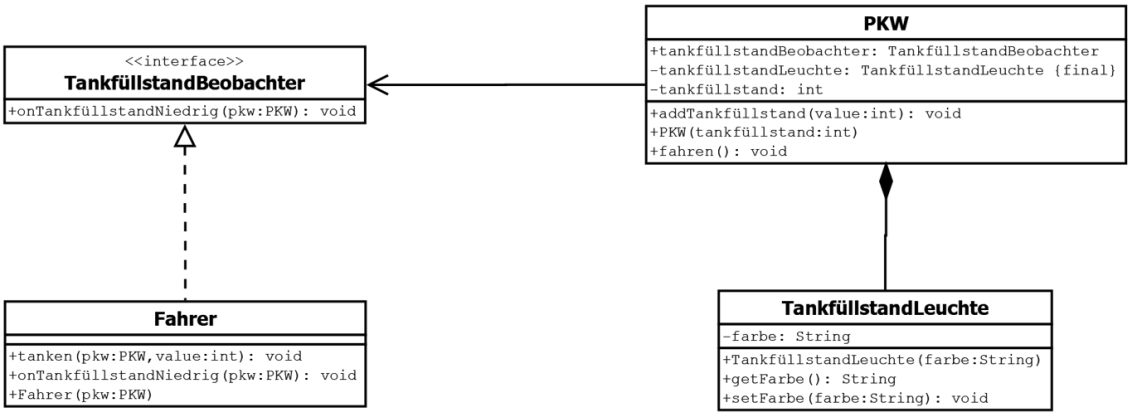
1.5 Übungsaufgabe

1.5.1 Aufgabenstellung

Erstellen Sie ein Klassendiagramm mit folgenden Komponenten:

1. Ein Interface namens `TankfüllstandBeobachter` mit einer `void`-Methode `onTankfüllstandNiedrig`, die ein `PKW`-Objekt als Parameter erhält.
2. Eine Klasse `TankfüllstandLeuchte` mit einem Attribut für die Farbe der Leuchte, den zugehörigen Getter- und Setter-Methoden und einem Konstruktor.
3. Eine Klasse `PKW` mit:
   - einer `TankfüllstandLeuchte`, die auf "Rot" gesetzt wird,
   - einem öffentlichen Attribut vom Typ `TankfüllstandBeobachter`,
   - einem privaten Attribut `tankfüllstand` und einer öffentlichen Methode `addTankfüllstand`, die den `tankfüllstand` erhöht und, wenn der `tankfüllstand` größer oder gleich 15 ist, die Farbe der Leuchte auf Grün setzt,
   - einem Konstruktor, der den `tankfüllstand` initialisiert,
   - einer `void`-Methode `fahren()`, die in einer Schleife "Brumm…" ausgibt und den `tankfüllstand` um 1 reduziert, solange der `tankfüllstand` größer 0 ist. Wenn der `tankfüllstand` kleiner 15 ist, stellt sie die Leuchte auf Gelb und ruft für das `TankfüllstandBeobachter`-Objekt die Methode `onTankfüllstandNiedrig` auf. Wenn der `tankfüllstand` kleiner 10 ist, stellt sie die Leuchte auf Rot und ruft ebenfalls die Methode `onTankfüllstandNiedrig` auf.
4. Eine Klasse `Fahrer`, die das Interface `TankfüllstandBeobachter` implementiert, mit:
   - einer Methode `tanken`, die den `tankfüllstand` eines übergebenen `PKW`-Objektes erhöhen kann,
   - der implementierten Interface-Methode `onTankfüllstandNiedrig`, die aus dem übergebenen `PKW` die Farbe der `TankfüllstandLeuchte` abfragt. Ist sie Gelb, sagt der Fahrer "Ach, hat noch Zeit…", ist sie Rot, sagt der Fahrer "Jetzt aber dringend tanken…" und tankt anschließend den `PKW`.

1.5.2 Lösung


<<interface>>
TankfüllstandBeobachter
+onTankfüllstandNiedrig(pkw:PKW): void
PKW
+tankfüllstandBeobachter: TankfüllstandBeobachter
-tankfüllstandLeuchte: TankfüllstandLeuchte {final}
-tankfüllstand: int
+addTankfüllstand(value:int): void
+PKW(tankfüllstand:int)
+fahren(): void
Fahrer
+tanken(pkw:PKW,value:int): void
+onTankfüllstandNiedrig(pkw:PKW): void
+Fahrer(pkw:PKW)
TankfüllstandLeuchte
-farbe: String
+TankfüllstandLeuchte(farbe:String)
+getFarbe(): String
+setFarbe(farbe:String): void